SIMATS ENGINEERING
ASSESSMENT TOOL 2

Course Name: Object Oriented Analysis and Design
Course Code: CSA11
Faculty In-Charge: Dr M. Ramamoorthy
Assessment: Annexure A - Technical Presentation
Total Marks: 40 | Weightage: 40%

Submitted By: K.Vijayalakshmi
Registration Number: 192511189

Code of Conduct

I,K.Vijayalakshmi(192511189), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Vijayalakshmi

1. Object-Oriented Design in Smart Healthcare Systems

Overview: A Smart Healthcare System is built to manage the day-to-day working of a hospital or clinic - keeping track of patients, doctors, appointments and medical records. Object-oriented programming fits this kind of system very well because a hospital is naturally made up of real-world entities that interact with each other, and OOP lets us model those entities directly as classes and objects.

Class and Object

A class is simply the blueprint of an entity, and an object is one real instance created from that blueprint. In this system, Patient, Doctor, Appointment and MedicalRecord are all classes. When a new person walks into the hospital and registers, the system creates a Patient object for them with their own patient ID, name and medical history. Every patient object follows the same Patient blueprint, but each one holds its own separate data, which is exactly why classes and objects make the system easy to scale as more patients join.

Encapsulation

Encapsulation means wrapping the data of an object together with the methods that operate on it, and restricting direct access to that data from outside the class. In a healthcare system this is extremely important because patient information is private and legally sensitive. The MedicalRecord class, for example, keeps fields like diagnosis and prescription as private, and only allows them to be read or changed through controlled methods such as addEntry() or encryptData(). This stops other parts of the program from directly editing sensitive medical data by mistake, and it gives the developer one single place to add validation or security checks.

Inheritance

Inheritance allows a new class to reuse the properties of an existing class. Here, both Patient and Doctor share common attributes such as name, age and contact number, so it makes sense to place these in a parent class called Person and let Patient and Doctor inherit from it. This avoids writing the same code twice, and if the hospital later decides to add a new field like emergency contact to Person, both Patient and Doctor automatically get it without any extra work.

Polymorphism

Polymorphism lets objects of different classes respond differently to the same method call. For example, a method called generateReport() could behave differently depending on whether it is called on a Patient object (producing a medical summary) or a Doctor object (producing a duty schedule). This means the

rest of the program can call generateReport() on any Person-type object without worrying about which exact subclass it is, which keeps the code flexible and easy to extend when new types of hospital staff are added later.

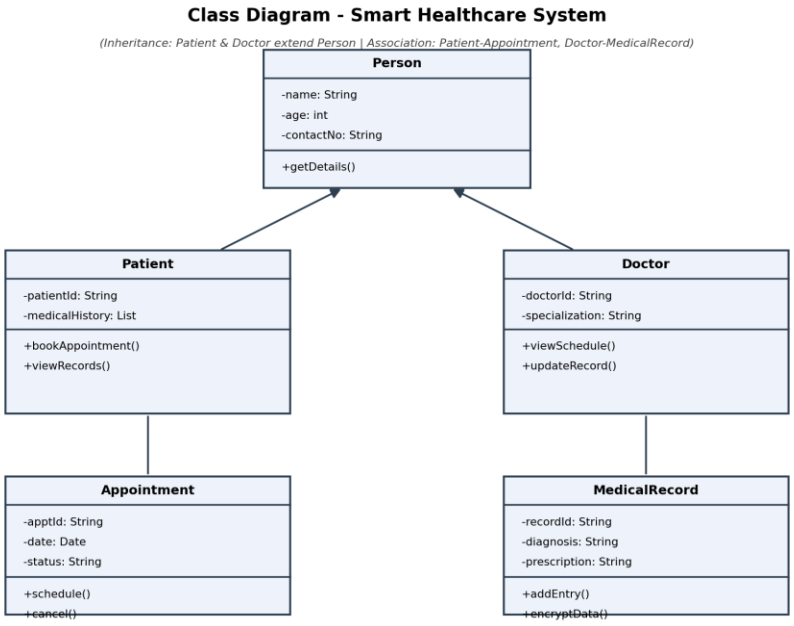

Fig 1.1 - Class diagram showing Person as the parent class, with Patient and Doctor inheriting from it

How These Concepts Improve the System

- Scalability - New entities such as Nurse or LabTechnician can be added by simply extending Person, without touching existing code.
- Security - Encapsulation keeps medical records and personal data hidden behind controlled methods, reducing the chance of accidental or unauthorized changes.
- Maintainability - Because responsibilities are divided cleanly between classes, a bug in the Appointment module can be fixed without disturbing the MedicalRecord module.

Overall, the four pillars of OOP work together here rather than separately - inheritance reduces repeated code, encapsulation protects sensitive fields, and polymorphism keeps the system open to future changes, which is exactly what a growing hospital application needs.

2. Applying Encapsulation and Abstraction in Online Banking

Overview: An Online Banking System has to handle fund transfers, account management and transaction tracking, and it has to do this while keeping customer money and personal information completely safe. Two OOP concepts do most of the heavy lifting here - abstraction and encapsulation - and even though people often use these two words interchangeably, they actually solve two different problems.

Abstraction

Abstraction is about hiding the complicated internal logic of a process and showing the user only what they need to see. When a customer transfers money using the banking app, they simply enter an amount and press transfer. They do not need to know that the system is validating the account balance, checking fraud rules, updating two different ledger tables and generating a transaction record behind the scenes. All of that complexity is hidden inside the transferFunds() method, and the customer only interacts with a simple interface.

Encapsulation

Encapsulation is about protecting the actual data. In the Account class, fields like balance, accountNumber and pin are kept private, meaning no other class in the system can reach in and change them directly. Instead, any change has to go through public methods such as deposit(), withdraw() or checkBalance(), which can include their own validation, for example rejecting a withdrawal that is larger than the current balance.

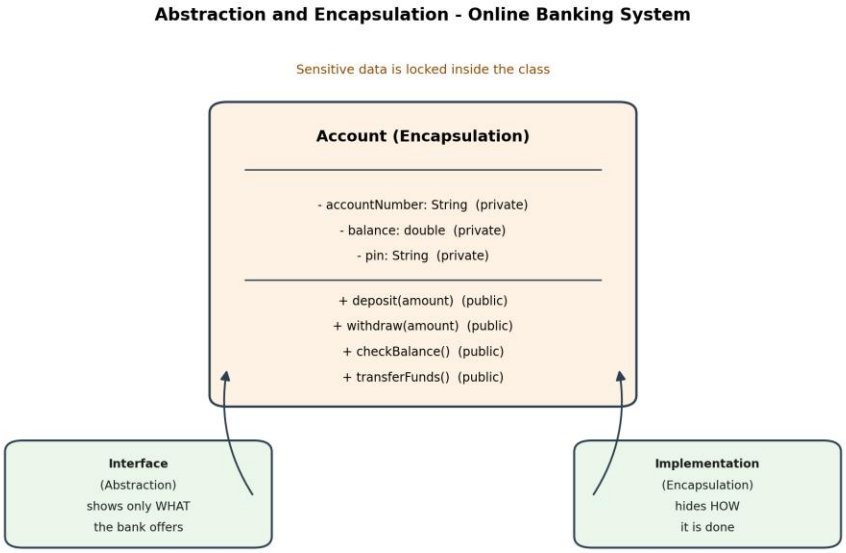

Fig 2.1 - Encapsulated Account class with private data and public controlled methods

Comparing Their Roles

Aspect	Abstraction	Encapsulation
Focus	Hides implementation complexity	Hides and protects the actual data
Level	Design level (what a class should do)	Code level (how the data is accessed)
Example in system	User only sees a Transfer button, not the internal steps	balance field is private and reachable only through deposit()/withdraw()
Main benefit	Simplifies how developers and users interact with the system	Prevents unauthorized or accidental modification of sensitive data

How This Helps Protect Data and Reduce Complexity

Because encapsulation keeps balance and pin private, even a programmer working on an unrelated part of the app, like the notifications module, cannot accidentally overwrite a customer's balance - they can only call the safe, validated methods provided by the Account class. Abstraction, on the other hand, means that if the bank later changes how transfers are processed internally, for instance by switching to a new payment gateway, the outer transferFunds() interface stays the same and nothing in the rest of the application needs to change. Together, abstraction keeps the system simple to use and easy to modify, while encapsulation keeps the sensitive financial data locked down and safe from misuse.

3. Modeling Object Relationships in a University Management System

Overview: A University Management System deals with students, faculty, departments and courses, and these entities do not exist in isolation - they are connected to each other in different ways. Choosing the right kind of relationship between classes is just as important as choosing the right classes themselves, because it decides how flexible and reusable the whole design will be.

Association

Association is the most general kind of relationship - it simply means that two classes are connected and use each other's services, but neither one owns the other or controls its lifetime. In this system, a Student is associated with an EnrollmentRecord: the student uses the enrollment record to keep track of which courses they are taking, but the two objects can be created, updated or deleted independently of each other.

Aggregation

Aggregation is a 'has-a' relationship where one class contains another, but the contained object can still exist on its own. A University has many Departments, but if a department were somehow removed from the university's list, the Department object itself would not automatically be destroyed - it could still exist, perhaps under a different university. This is a whole-part relationship with independent lifetimes.

Composition

Composition is a stronger form of the has-a relationship where the part cannot exist without the whole. A Department owns its Course objects, and if the department is removed from the system entirely, its courses do not make sense on their own and should be removed as well. This tight binding is what separates composition from aggregation.

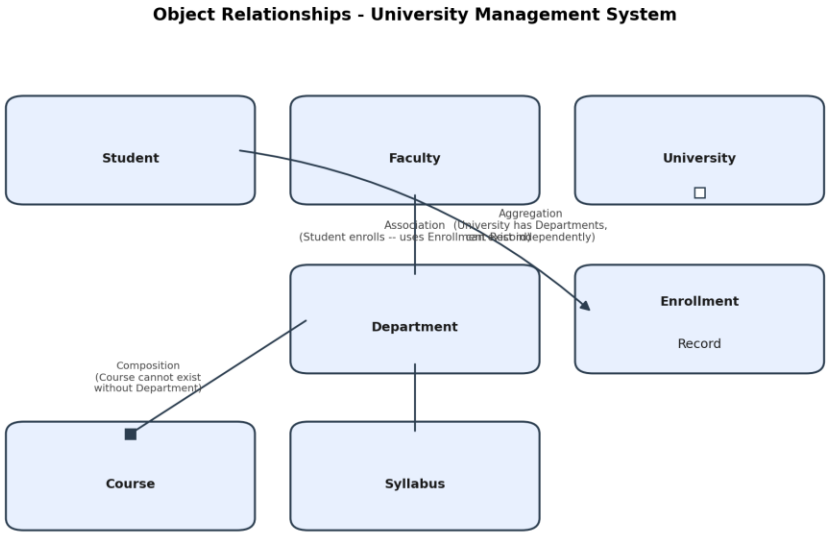

Fig 3.1 - Association, Aggregation and Composition modeled between University, Department, Course and Student

Why the Right Relationship Matters

- Flexibility - Using aggregation for University-Department means departments can be reassigned or reused without being tied to one single university object.
- Reusability - Association between Student and Course allows the same Course object to be linked to hundreds of different students without duplicating course data.
- Maintainability - Composition between Department and Course makes it clear in the code itself that courses are dependent data, so deleting a department safely cleans up its courses instead of leaving orphaned records in the database.

If every relationship in the system were modeled as a simple association, the design would technically work, but it would hide important rules about ownership and lifetime, and this usually leads to bugs later - for example, a deleted department leaving behind courses that no longer belong anywhere. Picking the correct relationship type at design time saves a lot of rework once the system is running in production.

4. Importance of SDLC in E-Commerce Development

Overview: An E-Commerce Platform needs to manage products, handle online payments and track orders reliably, often for thousands of users at the same time. Building something this critical without a structured process is risky, which is why the Software Development Life Cycle (SDLC) is followed - it breaks the entire project into clear phases so that nothing important gets missed.

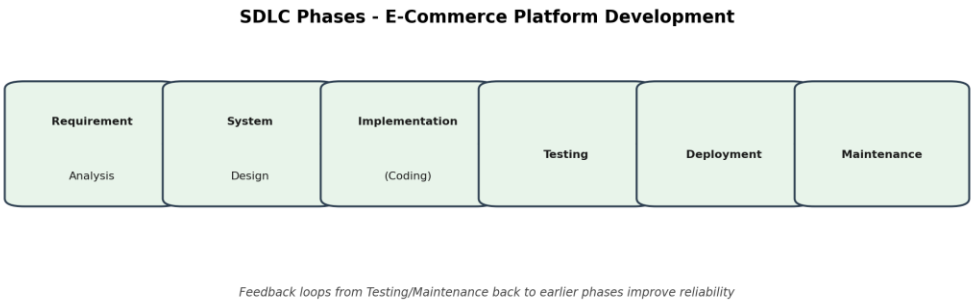

Fig 4.1 - The six phases of the SDLC applied to the e-commerce project

Phases of SDLC

- Requirement Analysis - The team gathers what the platform actually needs to do: product listings, cart, checkout, payment gateway integration, and order tracking. Getting this right early avoids expensive changes later.
- System Design - Architects decide the database structure, the class design for entities like Product, Order and Payment, and how the different modules will talk to each other.
- Implementation (Coding) - Developers write the actual code for each module, following the design created in the previous phase.
- Testing - The platform is tested for bugs, security holes and performance issues, especially around payment processing where mistakes are costly.
- Deployment - The tested platform is released to real users, often to a small group first before a full rollout.
- Maintenance - After launch, the team fixes bugs that appear in real use, adds new features, and keeps the system updated as customer needs change.

How Each Phase Contributes to Quality and Customer Satisfaction

Requirement analysis directly affects reliability, because a missed requirement, such as forgetting to handle failed payments, can cause serious problems after launch. System design affects scalability - a

well-designed database can handle a festive sale with a sudden spike in orders, while a poorly designed one may crash. Implementation quality shows up as fewer bugs and a smoother checkout experience, which builds customer trust. Testing is what catches issues such as a coupon code not applying correctly before customers ever see them, protecting the brand's reputation. Deployment done carefully, in stages, reduces the chance of a broken release affecting every user at once. Finally, maintenance keeps the platform relevant - fixing issues quickly and adding features customers ask for is often what keeps them coming back.

In short, SDLC is not just paperwork - each phase acts as a checkpoint that reduces risk, and skipping any one of them tends to show up later as bugs, security issues, or unhappy customers.

5. Selecting an Appropriate SDLC Model for Mobile App Development

Overview: A Mobile Learning Application is being built for a company where customer requirements change frequently - new content types, changing UI expectations, and features being added based on user feedback. In this situation, choosing the right SDLC model matters as much as choosing the right technology.

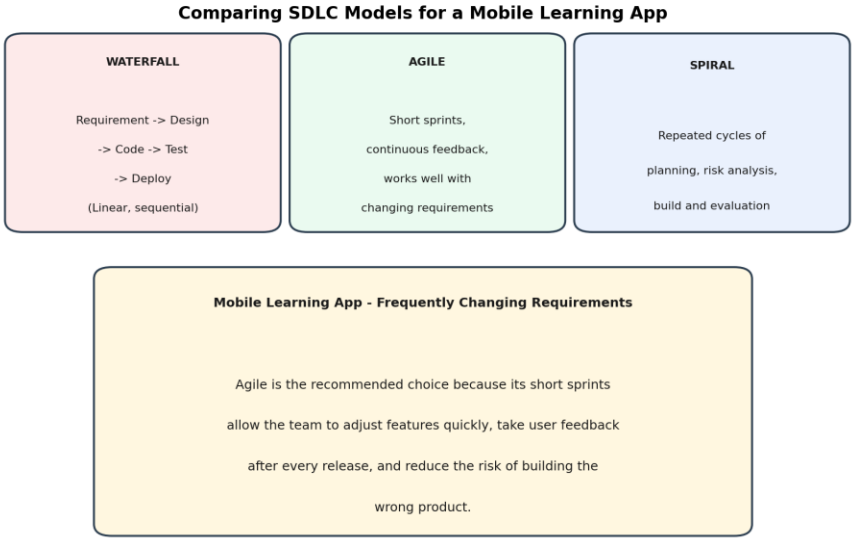

Fig 5.1 - Comparing Waterfall, Agile and Spiral for a requirement that keeps changing

Waterfall Model

Waterfall follows a strict sequence - requirements, design, coding, testing, deployment - where each phase must finish before the next one starts. It works well when requirements are fixed and clearly understood from day one. Its main weakness here is that if the client asks for a new feature midway through development, the whole plan has to be reopened, which is expensive and slow.

Agile Model

Agile breaks the project into short cycles called sprints, usually two to four weeks long, and delivers a working piece of the app at the end of each sprint. The team gets feedback constantly and can adjust direction quickly. This fits a mobile learning app very well because the client can try out each new version and request changes before too much work has gone into the wrong direction.

Spiral Model

Spiral combines iterative development with a strong focus on risk analysis at every cycle. It suits large, high-risk projects, such as a banking or defense system, where the cost of failure is very high and repeated risk checks are worth the extra time and documentation they need.

Comparison Table

Model	Advantage	Limitation
Waterfall	Simple to plan and manage; clear milestones	Very rigid; expensive to handle changing requirements
Agile	Handles changing requirements smoothly; frequent feedback	Needs an involved client and disciplined team communication
Spiral	Strong risk management for critical systems	Heavy documentation; not worth it for smaller projects

Recommendation

For this Mobile Learning Application, Agile is the most suitable choice. The company's requirements are described as changing frequently, which is exactly the situation Agile is built for - short sprints let the team show progress every few weeks, gather real feedback from users, and change course cheaply. Waterfall would force the team to lock requirements too early, and Spiral's heavy risk-analysis overhead is not justified for a learning app that is not handling high-risk operations like financial transactions.

6. Modular Design in a Hospital Management System

The Problem: The existing Hospital Management System is facing constant maintenance trouble because its modules - patient records, billing, pharmacy and appointments - are tightly coupled. In practice this means a small change in one module, such as updating how billing calculates a discount, ends up breaking something unrelated, like the pharmacy stock report. This happens because the modules directly depend on each other's internal code instead of talking through a clean, defined interface.

What Tight Coupling Looks Like Here

- The Billing module directly reads private fields from the Patient class instead of going through public methods.
- The Pharmacy module calls internal functions of the Appointment module to check if a patient is currently admitted, instead of using a shared, well-defined service.
- Any change to one class' internal structure forces developers to search through unrelated modules to check what might break.

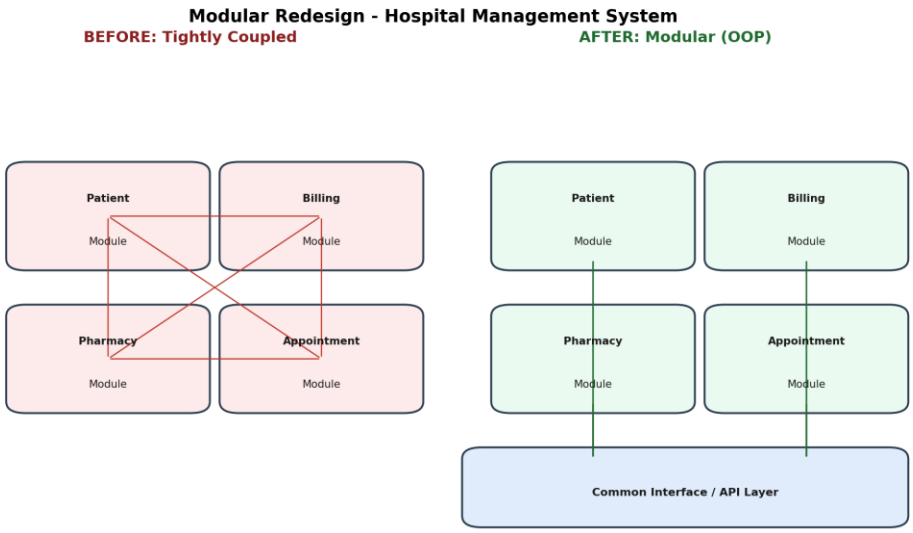

Fig 6.1 - Before: tightly coupled modules with direct dependencies. After: modular design connected through a common interface layer

Redesigning With Object-Oriented Principles

The fix is to apply proper encapsulation and clear interfaces between modules. Each module - Patient, Billing, Pharmacy, Appointment - should expose only a small set of public methods that other modules are allowed to call, while keeping their internal data private. For example, instead of Billing reading

Patient's private fields directly, it should call a method like patient.getBillingInfo(), which returns only the information billing actually needs. This way, the Patient class can change its internal storage at any time without breaking Billing, as long as getBillingInfo() keeps returning the same kind of result.

Introducing a shared interface layer, almost like a reception desk that every module talks through, also helps a lot. Instead of modules calling each other directly, they go through this common layer, which keeps dependencies organized and predictable.

Benefits of Modular Design

- Maintainability - Developers can safely change one module's internal logic, such as how the pharmacy checks stock, without worrying about breaking billing or appointments.
- Scalability - New modules, such as a Lab Reports module, can be added later by simply connecting to the shared interface, without redesigning existing modules.
- Code Reusability - A well-encapsulated Patient class can be reused as-is in a different hospital branch's system, since it does not carry hidden dependencies on other modules.

Overall, the redesign does not need new technology - it needs discipline in how classes expose their data to each other, which is really just encapsulation and interface-based design applied consistently across the whole system.

7. Enhancing Software Quality Using Design Patterns

The Problem: The Hotel Reservation System currently suffers from duplicated code - for example, the logic to create a DeluxeRoom object and a StandardRoom object is copy-pasted in several places with only small differences - and from inefficient object creation, where multiple parts of the system each create their own separate ReservationManager, leading to conflicting booking data.

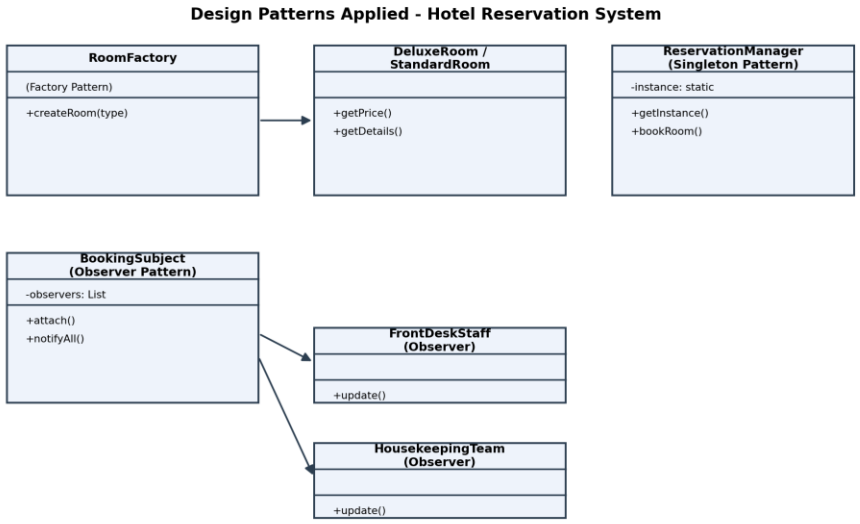

Fig 7.1 - Factory, Singleton and Observer patterns applied to the hotel reservation system

Factory Pattern

The Factory Pattern centralizes object creation in one place. Instead of scattering room-creation logic across the codebase, a single RoomFactory class decides which type of room object to create based on the input it receives, for example createRoom("deluxe") or createRoom("standard"). This removes duplicated creation code and makes it easy to add a new room type later - the rest of the system does not need to change, only the factory does.

Singleton Pattern

The Singleton Pattern makes sure a class has exactly one instance throughout the whole application. Applying this to ReservationManager guarantees that every part of the system - the website, the mobile app, and the front-desk software - shares the same booking data instead of each one keeping its own separate, and possibly conflicting, copy. This directly solves the inefficient object creation problem mentioned in the case.

Observer Pattern

The Observer Pattern lets one object (the subject) automatically notify a list of other objects (the observers) whenever something changes. Here, a BookingSubject can notify both the FrontDeskStaff and the HousekeepingTeam the moment a new reservation is made, so housekeeping knows to prepare a room without anyone having to send a manual message.

How These Patterns Improve the System

- Flexibility - New room types or new staff roles that need booking notifications can be added without changing existing classes.
- Maintainability - Room-creation logic lives in one factory class instead of being duplicated everywhere, so a bug fix only has to be made once.
- Software Quality - The Singleton pattern removes a whole category of bugs caused by conflicting reservation data, which used to be a common source of double-booking complaints.

Design patterns are really just proven solutions to problems that come up again and again in software design. Applying Factory, Singleton and Observer here does not add unnecessary complexity - it actually removes the mess that was already there from repeated, ad-hoc code.

8. UML-Based Modeling for an Online Examination System

Overview: An Online Examination System manages students, instructors, examinations and result processing. Before writing a single line of code, it helps to visualize the system using UML (Unified Modeling Language) diagrams, because they let everyone involved - developers, instructors and even non-technical stakeholders - see how the system will work and agree on it early.

1. Use Case Diagram

The use case diagram shows the system from the point of view of its users. Here there are two main actors: the Student, who can log in, take an exam and view their result, and the Instructor, who can create an exam, grade answers and manage the question bank. Drawing this diagram early makes it very easy to spot missing functionality - for instance, without this diagram, the team might forget to design a 'view result' feature for students.

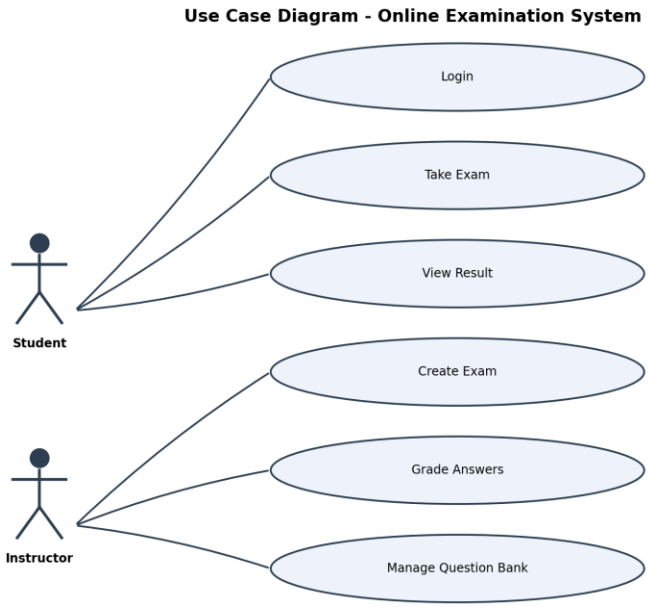

Fig 8.1 - Use Case Diagram showing Student and Instructor interactions

2. Class Diagram

The class diagram shows the static structure of the system - the actual classes, their attributes and their relationships. In this system, Student, Instructor, Exam, Question and Result are the core classes. An Exam is made up of several Question objects, and completing an Exam produces a Result object. This diagram becomes the direct blueprint for the code that developers will write.

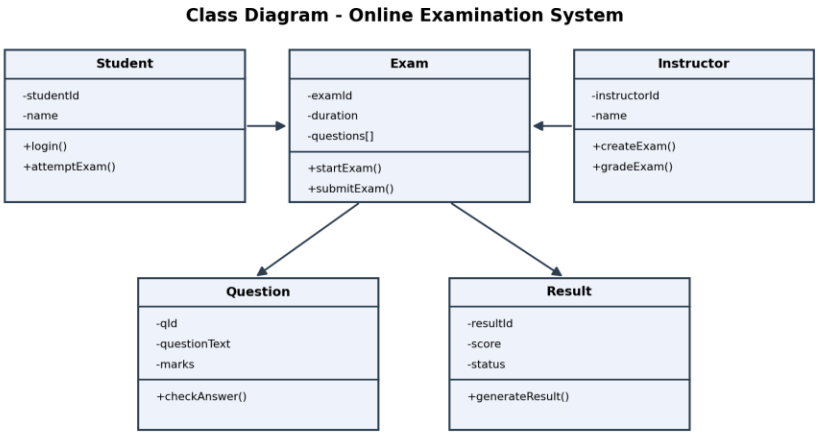

Fig 8.2 - Class Diagram showing Student, Instructor, Exam, Question and Result and how they connect

3. Activity Diagram

The activity diagram shows the flow of actions over time - what happens step by step when a student takes an exam, starting from login, moving through answering questions and submitting the exam, and ending with the result being generated and shown. This is useful for spotting logical gaps, such as what should happen if the student's time runs out before they submit.

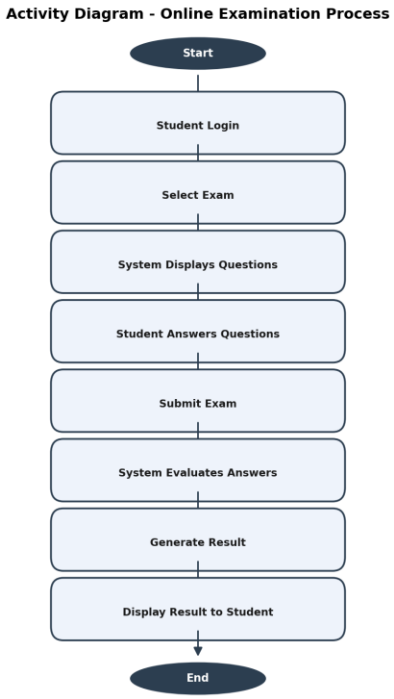

Fig 8.3 - Activity Diagram showing the step-by-step exam process

Why UML Modeling Helps

- System Visualization - Diagrams give everyone a shared picture of the system instead of relying on scattered notes or verbal descriptions.
- Requirement Analysis - Drawing the use case diagram often surfaces requirements that were missed in the initial discussion, like the need for a question bank management feature.
- Stakeholder Communication - An instructor who has never written code can still look at the use case diagram and confirm that it matches what they actually need, without having to read any code.

Put together, these three diagrams cover three different questions about the same system: the use case diagram answers 'who can do what', the class diagram answers 'what does the system look like internally', and the activity diagram answers 'what actually happens, step by step'. Using all three during design catches far more mistakes early than trying to jump straight into coding.